

Corso di Laurea Magistrale in Data Science and Business
Informatics

Business Process Modeling

Progetto n. 72

Centro Termale

Alice Calderini - 634459

Anno Accademico 2025/2026

1 Introduzione

L'obiettivo di questo progetto è modellare i processi che rappresentano uno scenario in cui due amici, Alice e Brian, desiderano organizzare una giornata presso un centro termale.

Alice è responsabile dell'organizzazione e avvia il processo, inviando a Brian un elenco di date in cui è disponibile. A questo punto Brian può: proporre una data e i servizi termali da prenotare; delegare ad Alice la scelta; oppure comunicare di non poter partecipare, concludendo così il processo.

Se Brian formula una proposta, Alice può accettarla oppure presentare un'alternativa. Nel caso in cui Brian preferisca non scegliere, sarà Alice a inviare una proposta iniziale. In seguito alla proposta ricevuta, Brian può confermare la scelta, avanzare una controproposta oppure annullare l'appuntamento, concludendo il processo.

Quando viene raggiunto un accordo, Alice procede con la prenotazione dell'ingresso al centro termale e dei trattamenti selezionati, quindi comunica a Brian l'orario e il luogo dell'incontro e il processo si conclude.

Vi è inoltre la possibilità che, durante la fase di preparazione, emergano problemi di disponibilità dei servizi termali o chiusure impreviste della struttura che rendano necessario rinviare la visita. In questo caso, Alice e Brian devono ripetere il processo decisionale fino al raggiungimento di un nuovo accordo.

Questa relazione descrive come sono stati modellati i processi, e presenta le caratteristiche e proprietà delle workflow net corrispondenti. Per la progettazione a livello più astratto è stata utilizzata la notazione Business Process Model and Notation (BPMN), in quanto consente di modellare sia *orchestration* che *collaboration* tra gli attori presenti. I diagrammi BPMN sono stati progettati con il tool online bpmn.io e convertiti in Petri nets con il tool online bpmn2petrinet.com. Tali reti sono state analizzate con WoPeD.

2 Diagrammi BPMN

Nello scenario si possono identificare due attori distinti: Alice e Brian. Sono stati creati, uno per ciascun attore, due diagrammi di *orchestration*, i quali interagiscono tra di loro in un diagramma di *collaboration*.

2.1 Alice

Il processo inizia (*start event*) e, come da specifiche, Alice invia un elenco delle proprie disponibilità con un *send task* "Invia disponibilità", a cui è associato un data object "Date disponibili", che rappresenta l'elenco delle disponibilità inviate da Alice.

A questo punto Alice resta in attesa della risposta di Brian. Poiché la scelta successiva dipende da un evento esterno, ossia dalla risposta di Brian, viene utilizzato un *event-based gateway*. In base all'evento che si verifica, il flusso può diramarsi in tre possibili *message intermediate catch events* che Alice può ricevere: "Brian non disponibile", "Riceve delega" e "Riceve proposta".

- Se Alice riceve non disponibilità da parte di Brian, allora il processo termina con un *end event*.

- Se Alice riceve la delega di effettuare una proposta, allora la invia a Brian con un *send task* “Invia proposta data, servizi”, a cui è associato il data object “Proposte”, contenente appunto la proposta inviata.
- Infine, se Alice riceve una proposta, ha due alternative (*XOR-split* “Accetta la proposta?”). Può accettarla comunicandolo a Brian (*send task* “Accetta”), e l’accordo si considera raggiunto. Di conseguenza, non è necessaria una ulteriore risposta da parte di Brian; pertanto, dopo il task “Accetta” il flusso prosegue verso lo *XOR-join* che conduce alla fase di prenotazione. In generale, l’accordo si considera raggiunto quando uno dei due partecipanti formula una proposta o una controproposta e l’altro la accetta. Alice però potrebbe non accettare la risposta di Brian e proporre un’alternativa, quindi con il *send task* “Invia controproposta”, a cui è associato il data object “Proposte[modificato]”, invia la sua scelta a Brian. Con uno *XOR-join* viene ricongiunto questo ramo con quello proveniente dal task “Invia proposta”.

Se Alice ha inviato una proposta o una controproposta a Brian, allora resta in attesa della sua risposta, e con un *event-based gateway* si modellano i tre diversi *message intermediate catch events* che Alice può ricevere: “Brian non disponibile”, “Accordo raggiunto”, “Riceve controproposta”.

- Se Alice riceve non disponibilità da parte di Brian, allora il processo termina con un *end event*.
- Se da parte di Brian riceve il messaggio di accettazione, l’accordo viene raggiunto e si passa alla fase di prenotazione. In particolare, tramite lo *XOR-join* successivo all’evento in questione “Accordo raggiunto”, il flusso proveniente da questo evento si ricongiunge con quello che arriva dal task “Accetta” descritto precedentemente. I due rami alternativi vengono quindi unificati in un unico flusso di controllo che dopo l’accordo prosegue verso la fase di prenotazione.
- Infine, se Alice riceve una controproposta ritorna alla fase di negoziazione della proposta di Brian, dove può accettarla e raggiungere un accordo oppure rifiutarla e proseguire con ulteriori trattative. A livello di diagramma, questo comportamento è modellato collegando l’evento “Riceve controproposta” allo *XOR-join* immediatamente seguito dallo *XOR-split* “Accetta la proposta?”, che rappresenta appunto le due alternative che Alice ha davanti: accettare la proposta di Brian oppure formulare una controproposta.

Dopo aver raggiunto l’accordo, Alice prenota l’ingresso al centro termale e i trattamenti selezionati (task “Prenota ingresso, servizi”), comunica a Brian la prenotazione (*send task* “Invia prenotazione” con data object “Prenotazione”) e il processo si conclude con un *end event*. Durante la prenotazione possono emergere degli imprevisti, come la chiusura del centro termale o la non disponibilità dei servizi. Nel diagramma questo è stato realizzato decorando il task “Prenota ingresso, servizi” con un *error boundary event*, che intercetta l’errore se questo si verifica durante la prenotazione. L’errore viene quindi comunicato a Brian tramite il *send task* “Comunica imprevisto”. Successivamente, il processo viene reindirizzato alla fase iniziale di negoziazione, tornando allo *XOR-join* immediatamente successivo allo *start event*.

Il diagramma di Alice è mostrato in Figura 1.

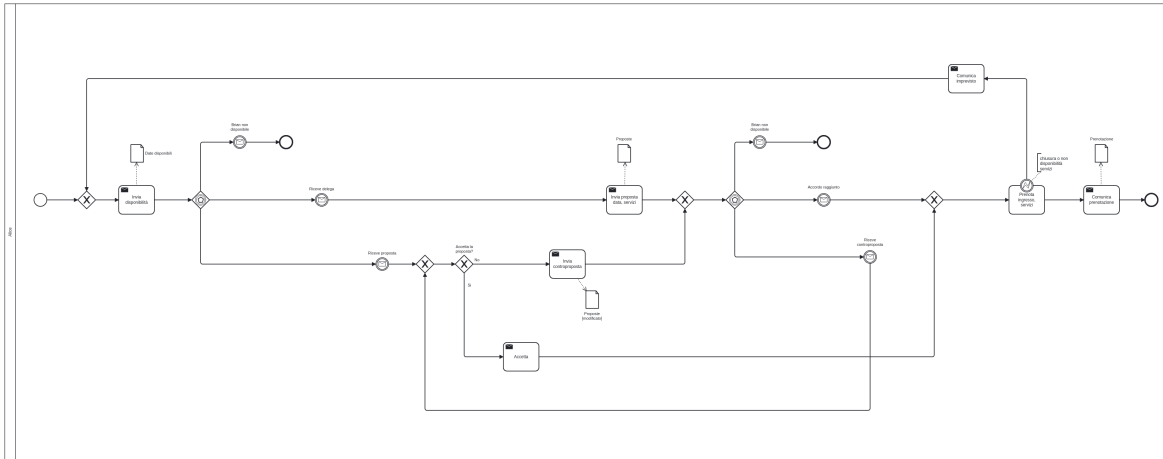


Figura 1: Diagramma BPMN di Alice

2.2 Brian

Il processo inizia (*start event*) e Brian riceve da Alice le date in cui lei è disponibile (con il *message intermediate catch event* “Riceve disponibilità”).

A questo punto, Brian ha tre possibilità di scelta, modellate con uno *XOR-split*:

- Brian comunica ad Alice che lui non è disponibile (*send task* “Comunica non disponibilità”) e il processo termina (*end event*).
- Brian può comunicare ad Alice che sia lei a scegliere la data e i servizi (*send task* “Delega scelta”). Questo task è seguito dal *message intermediate catch event* “Riceve proposta” contenente la scelta fatta da Alice.
- Brian può scegliere autonomamente data e servizi e comunicare la sua proposta ad Alice (*send task* “Invia proposta data, servizi” con data object “Proposte”). Brian rimane quindi in attesa di una risposta da parte di Alice, gestita tramite un *event-based gateway*:
 - Brian riceve un messaggio da parte di Alice che l’accordo è stato raggiunto (*message intermediate catch event* “Accordo raggiunto”).
 - Brian riceve una controproposta da parte di Alice (*message intermediate catch event* “Riceve controproposta”). Successivamente con uno *XOR-join* si ricongiunge questo ramo a quello proveniente dall’evento “Riceve proposta”.

Quindi, se Brian ha ricevuto una proposta o una controproposta ha tre diverse possibilità (modellate con uno *XOR-split*):

- Può comunicare ad Alice che annullerà l’appuntamento (*send task* “Rifiuta”) e il processo si conclude con un *end event*.
- Può inoltre formulare una controproposta comunicandola ad Alice tramite il *send task* “Invia controproposta” con data object “Proposte [modificato]”. A livello di diagramma, il flusso viene collegato allo *XOR-join* immediatamente successivo al task “Invia proposta data, servizi”, in modo da riportare il processo alla fase di

negoziiazione. Brian torna quindi in attesa (*event-based gateway*) di una risposta da parte di Alice, che può essere, come già descritto, un messaggio di accordo raggiunto oppure una nuova controproposta.

- Può comunicare ad Alice che accetta la proposta (*send task* “Accetta”). Successivamente con uno *XOR-join* viene ricongiunto questo ramo con quello proveniente dall’evento precedentemente descritto “Accordo raggiunto”.

Una volta raggiunto l’accordo, un *event-based gateway* modella le varie possibilità di Brian in base a ciò che comunica Alice.

- Se Brian riceve la prenotazione (*message intermediate catch event* “Riceve prenotazione”), il processo si conclude con un *end event*.
- Se Brian riceve la comunicazione di un imprevisto (tramite il *message intermediate catch event* “Riceve comunicazione imprevisto”), il flusso del processo ritorna alla fase iniziale di negoziazione, nello *XOR-join* subito successivo allo *start event*.

Il diagramma di Brian è rappresentato in Figura 2

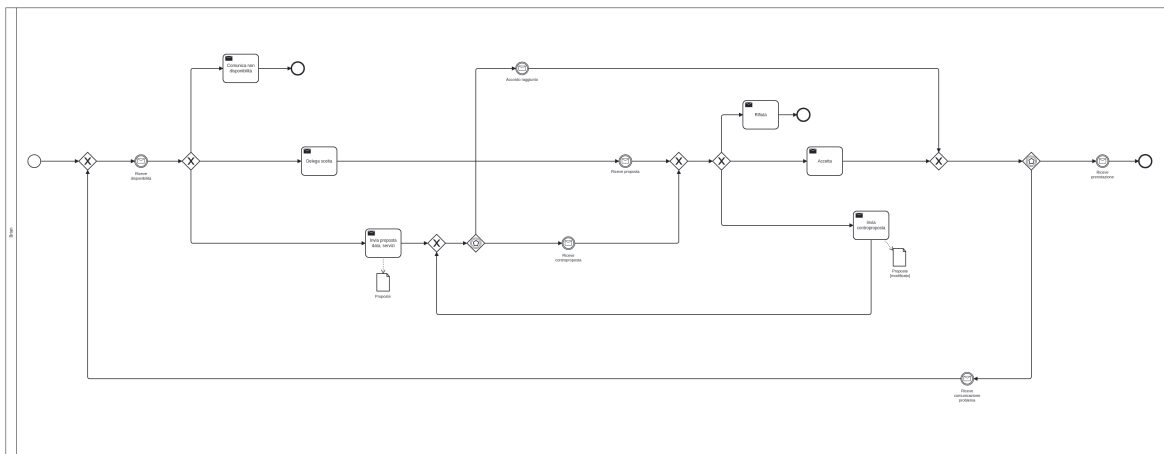


Figura 2: Diagramma BPMN di Brian

2.3 Collaboration

Il diagramma di *collaboration* è composto dai singoli diagrammi di *orchestration* Alice e Brian, collegati tra loro tramite *message flows* tra i *send task* e i *receive event* corrispondenti. Il diagramma è mostrato in Figura 3

3 Workflow nets

I diagrammi BPMN sono stati tradotti in workflow nets, al fine di poter condurre un’analisi formale delle loro proprietà. Ciascun diagramma, sia i due di *orchestration* sia quello di *collaboration*, è stato convertito in una workflow net utilizzando il tool bpmn2petrinet.com.

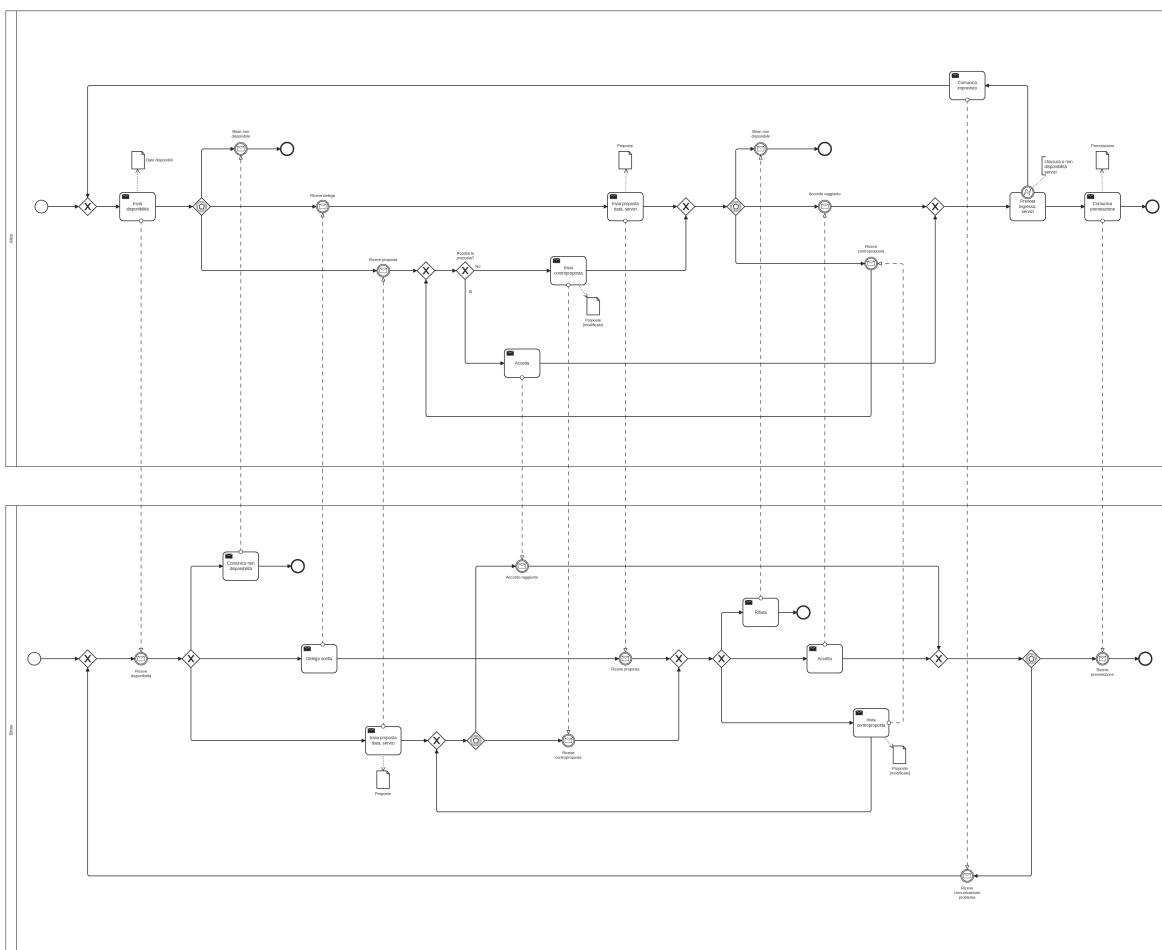


Figura 3: Diagramma BPMN della collaborazione tra Alice e Brian

È opportuno specificare che il tool non è in grado di gestire l'*error boundary event*; pertanto, per poter effettuare la conversione in Petri net, è stato necessario modificare il diagramma di Alice (e di conseguenza anche quello di *collaboration*), mantenendo per quanto possibile la semantica dell'*error boundary event*.

In particolare, è stata modificata solamente l'ultima parte del diagramma: è stato introdotto uno *XOR-split* denominato “Imprevisti?”, che presenta rami differenti a seconda dell'esito della prenotazione.

Le alternative dello *XOR-split* sono:

- “No”: Alice procede con la prenotazione tramite il task “Prenotazione ingresso, servizi” e, come nella versione precedente del modello, tramite il *send task* “Invia prenotazione” invia a Brian la prenotazione. Il processo si conclude quindi con un *end event*.
- “Chiusura o non disponibilità servizi”: rappresenta il caso di errore. Analogamente alla versione precedente, tramite il *send task* “Comunica imprevisto” Alice informa Brian dell’imprevisto e il processo ritorna alla fase iniziale delle trattative.

Il diagramma di Brian non ha subito modifiche, mentre il diagramma modificato di Alice è riportato in Figura 4.

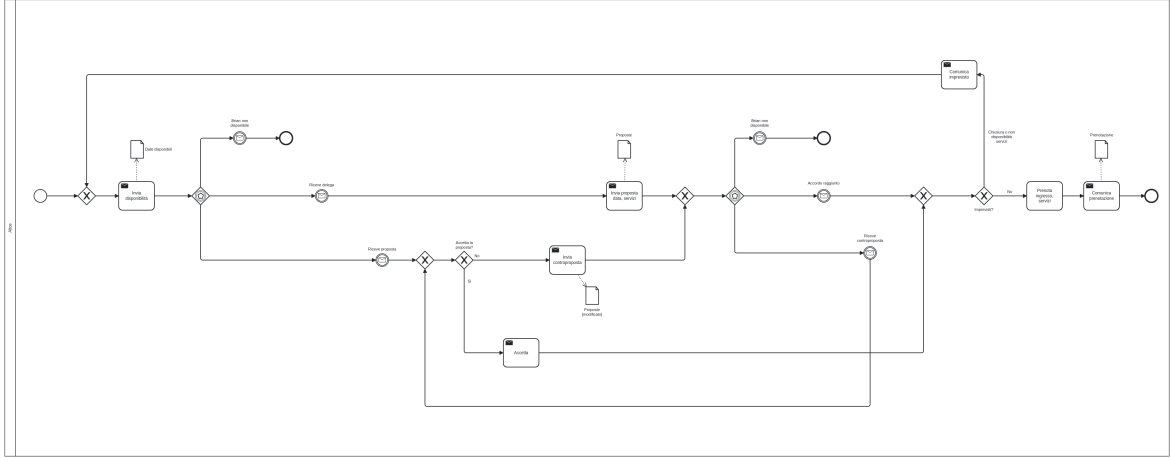


Figura 4: Diagramma BPMN di Alice per conversione in workflow net

3.1 Alice

La rete di Petri di Alice è mostrata in Figura 5.

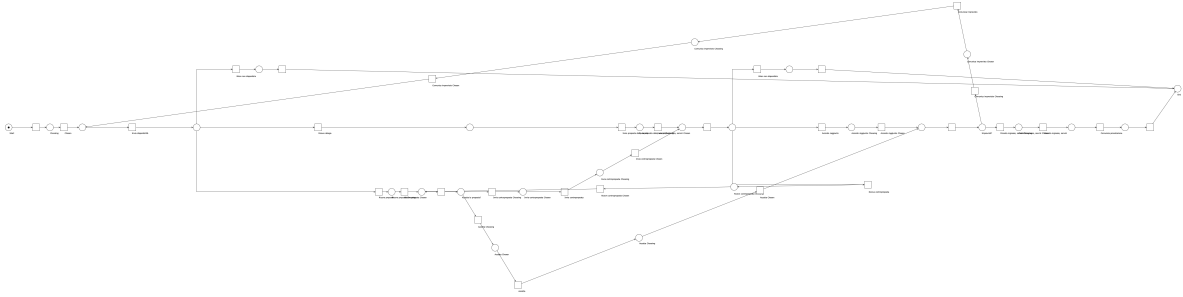


Figura 5: Workflow net del processo di Alice

La rete è composta da 27 posti, 32 transizioni e 64 archi. Soddisfa le proprietà di workflow net: esiste un unico posto iniziale i , $\bullet i = \emptyset$ e un unico posto finale o , $o \bullet = \emptyset$; e ogni altro posto o transizione si trova su un cammino da i a o . Tutte le transizioni hanno preset e postset non vuoti e N^* è composta da un'unica componente connessa (in particolare, fortemente connessa). La rete non presenta violazioni della proprietà di free-choice ed è well-structured, quindi N^* è well-handled, cioè non presenta né TP-handles né PT-handles. N^* non è un T-system, tuttavia è S-coverable e contiene una S-component che copre tutti i posti della rete. Ciò implica che N^* coincida con tale S-component, ovvero è una S-net, in particolare un S-system. Quindi la workflow net N è anch'essa un S-system. Di conseguenza, è safe e sound. Ne segue direttamente che N^* sia bounded e live. Inoltre, un qualsiasi vettore $I = [k, \dots, k] \in \mathbb{Q}^{27}$ è un S-invariant della workflow net N . Inoltre, dato che N^* è live e bounded, ha un T-invariant positivo. Il reachability graph ha 27 nodi e 33 archi. La rete è bounded, quindi il suo reachability graph è finito e coincide con il coverability graph.

3.2 Brian

La rete di Petri di Brian è mostrata in Figura 6.

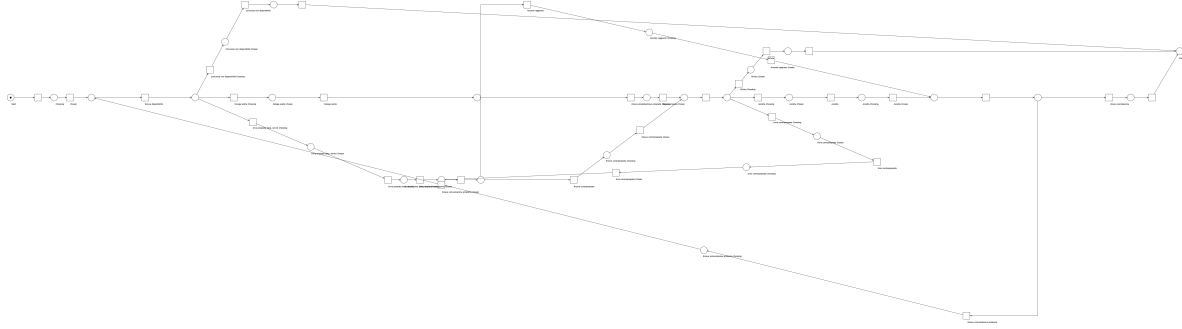


Figura 6: Workflow net del processo di Brian

La rete è composta da 28 posti, 33 transizioni e 66 archi. La rete soddisfa le proprietà di workflow net, è free-choice ed è well-structured. N^* non è un T-system, tuttavia è S-coverable e contiene una S-component, quindi la rete è un S-system. Di conseguenza, la rete è safe e sound e N^* è live e bounded. Inoltre, dato che N^* è live e bounded, ha un T-invariant positivo. Il reachability graph ha 28 nodi e 34 archi. La rete è bounded, quindi il suo reachability graph è finito e coincide con il coverability graph.

3.3 Workflow system

Il workflow system è mostrato in Figura 7.

La rete è composta da 69 posti, 67 transizioni e 160 archi. Non è free-choice, infatti presenta 4 *free choice violations* dovuti alla presenza dei 4 *event-based gateways*. Inoltre non è well-structured, infatti presenta 104 PT-Handles e 106 TP-Handles. N^* è fortemente connessa; inoltre è S-coverable, con 16 S-components distinte, quindi ha un S-invariant positivo. La rete soddisfa le proprietà di workflow net ed è sound. Inoltre, dato che N^* è live e bounded, ha un T-invariant positivo. Il reachability graph ha circa 180 nodi e 450 archi. La rete è bounded, quindi il suo reachability graph è finito e coincide con il coverability graph.

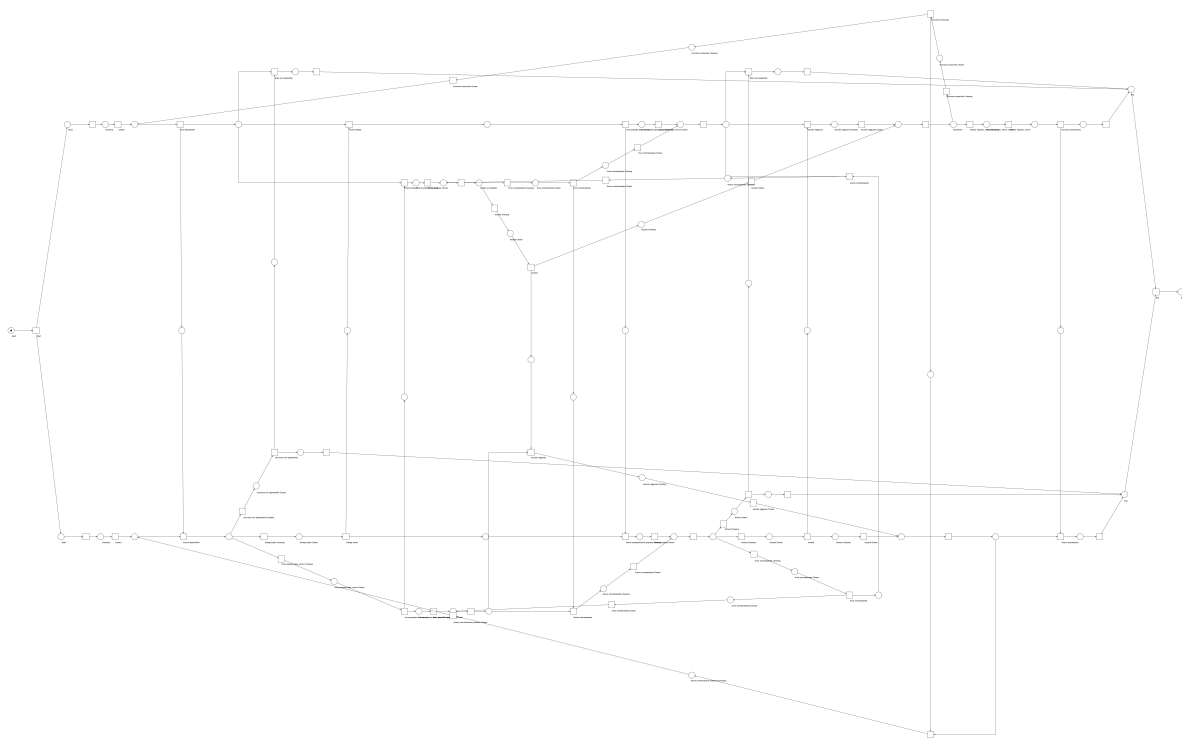


Figura 7: Workflow net della collaborazione